

# 05 - Object Oriented Programming

哈尔滨工业大学 黄子扬

2026 年 6 月 6 日

## Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Why OOP? / 为什么需要面向对象?</b>                         | <b>2</b> |
| <b>2</b> | <b>Creating Classes / 创建类</b>                        | <b>2</b> |
| 2.1      | The Blueprint / 蓝图定义 . . . . .                       | 2        |
| 2.2      | The <code>__init__()</code> Method / 初始化方法 . . . . . | 2        |
| <b>3</b> | <b>Attributes / 属性</b>                               | <b>2</b> |
| <b>4</b> | <b>Instantiating &amp; Methods / 实例化与方法</b>          | <b>3</b> |
| 4.1      | Instantiating / 实例化对象 . . . . .                      | 3        |
| 4.2      | Instance Methods / 实例方法 . . . . .                    | 3        |
| 4.3      | Dunder Methods / 魔术方法 . . . . .                      | 3        |
| <b>5</b> | <b>Inheritance / 继承</b>                              | <b>3</b> |
| <b>6</b> | <b>Encapsulation &amp; Properties / 封装与属性装饰器</b>     | <b>4</b> |

# 1 Why OOP? / 为什么需要面向对象？

## 【Concept / 核心概念】1

From Lists to Classes / 从列表到类 Imagine tracking employees: `Eric = ["Eric", 34, "CEO"]` [1]. If "Age" is missing for Alice, the index positions shift and your code breaks [2]. Classes allow you to create **User-defined Data Structures** that group data and behaviors together [2].

想象一下用列表存储员工信息。如果某人漏填了年龄，列表索引就会乱序，导致程序出错。**\*\* 类 (Class)\*\*** 允许你自定义数据结构，将数据和行为打包在一起。

## 2 Creating Classes / 创建类

### 2.1 The Blueprint / 蓝图定义

- **Keyword:** Use `class` followed by the name. / 使用 `class` 关键字 [2].
- **Naming:** Use **CamelCase** (e.g., `Employee`), starting with a capital letter. / 类名使用大驼峰命名法 [3].
- **pass:** A placeholder used as a body where code will eventually go. / 占位符，防止空类报错 [2].

### 2.2 The `__init__()` Method / 初始化方法

## 【Concept / 核心概念】1

The Initializer This method runs every time a new object is created to set its initial state (values). / 每次创建新对象时，此方法会自动运行，设置对象的初始值 [3].

## 【Warning / 避坑指南】1

What is `self`? / `self` 到底是什么？The first argument of `__init__()` is **always** `self`. It is a reference to the specific instance being created. Think of it as a **placeholder** while building the blueprint [4]. `self` 指代“当前的这个实例”。它是构建蓝图时的占位符。

## 3 Attributes / 属性

- **Instance Attributes:** Belong to a specific object (e.g., `self.name`). / **实例属性**：每个对象独有的数据 [4].
- **Class Attributes:** Same for ALL instances of a class (e.g., `company_name = "Emlyon"`). / **类属性**：所有对象共享的数据 [5].

### 【Example / 实操案例】1

Class Definition Example / 类定义完整案例

```
class Employee:
    company_name = "Emlyon" # Class Attribute [5]

    def __init__(self, name, age):
        self.name = name # Instance Attribute [4]
        self.age = age # Instance Attribute
```

## 4 Instantiating & Methods / 实例化与方法

### 4.1 Instantiating / 实例化对象

#### 【Example / 实操案例】1

Creating Objects / 创建对象实操

```
emp1 = Employee("David", 23) # Instantiate an object [5]
print(emp1.name) # Access using dot notation [6]
emp1.age = 36 # Attributes are MUTABLE by default [6]
```

### 4.2 Instance Methods / 实例方法

#### 【Concept / 核心概念】1

Behaviors / 行为 Functions inside a class that define actions an object can perform. The first argument is always `self` [7].

类内部的函数，定义了对象能做的动作。

### 4.3 Dunder Methods / 魔术方法

- `__str__()`: Changes what gets displayed when you `print()` the object [7]. / 定义打印对象时的显示内容。

## 5 Inheritance / 继承

#### 【Concept / 核心概念】1

Parent vs. Child / 父类与子类 Inheritance is the process where one class takes on the attributes and methods of another [8]. Child classes can **override** or **extend** the parent's logic [9].

继承允许一个类获取另一个类的所有特征。子类可以重写或扩展父类的逻辑。

### 【Example / 实操案例】 1

The `super()` keyword / `super()` 的妙用 Use `super()` to call the parent class's methods, especially the `__init__` [10].

```
class HREmployee(Employee): # Inherit from Employee [10]
    def __init__(self, name, age, wh):
        super().__init__(name, age) # Initialize parent attributes [10]
        self.working_hours = wh

    def __str__(self): # Override string representation [10]
        return f"{self.name} works in HR with {self.working_hours} hours."
```

## 6 Encapsulation & Properties / 封装与属性装饰器

### 【Concept / 核心概念】 1

Read-only Attributes / 只读属性 Use `@property` to make an attribute read-only. It uses a "getter-function" to return the value but prevents direct assignment from outside [11].

使用 `@property` 装饰器可以将方法伪装成属性，实现只读访问，防止外部非法篡改 [11]。

### 【Example / 实操案例】 1

Property 案例

```
class MyClass:
    def __init__(self):
        self._hidden = 42 # Prefix '_' suggests internal use

    @property
    def hidden(self): # Getter function [12]
        return self._hidden

my_obj = MyClass()
print(my_obj.hidden) # Works like an attribute [12]
# my_obj.hidden = 0 # This would raise an error! [12]
```